

Backend - Contact API (Hidrorural)

Breve guía para dejar operativo el envío de correos desde la sección **Cotización** de la web.

Resumen

- Servidor: Node.js + Express
- Envío de email: **nodemailer** usando SMTP (actualmente Gmail App Password en **.env**)
- Endpoint: **POST /api/contact** (JSON)
- Protecciones: honeypot (**hp**) y rate-limit

Variables de entorno (archivo **.env**)

Crear **backend/server/.env** con las siguientes variables (no subir a VCS):

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=tu.cuenta@gmail.com
SMTP_PASS=TU_APP_PASSWORD_SIN_ESPACIOS
SMTP_SECURE=false
FROM_EMAIL="Hidrorural <tu.cuenta@gmail.com>"
TO_EMAIL=contacto@hidrorural.com
FRONTEND_ORIGIN=http://localhost:5173
PORT=4000

# Opcionales: copiar a otros destinatarios
CC_EMAIL=otra@dominio.com
BCC_EMAIL=oculto@dominio.com
```

Notas:

- **SMTP_PASS** debe ser un App Password de Google (sin espacios). Nunca pongas contraseñas reales en el repositorio.
- Para múltiples direcciones separa con comas (ej. **a@x.com,b@y.com**) — **nodemailer** acepta la cadena con comas.

Pasos para generar App Password en Gmail

1. Accedé a tu cuenta Google (la que usás en **SMTP_USER**).
2. Activá la Verificación en dos pasos (2-Step Verification) si no está habilitada: Account → Security → 2-Step Verification.
3. En Account → Security → App passwords: crear una nueva contraseña de aplicación.
 - Seleccioná **Mail** como app y **Other** o el dispositivo que prefieras, dale un nombre (p.e. **hidrorural-server**).
4. Google mostrará una contraseña de 16 caracteres agrupada con espacios; copiala y pegala en **.env** sin espacios, por ejemplo:
 - si Google muestra **abcd efgh ijkl mnop**, en **.env** usar **SMTP_PASS=abcdefghijklmnop**.

Ejecutar servidor en local

```
cd backend/server
npm install
node server.js
```

La salida debe mostrar: `SMTP transporter OK` y `Contact server listening on 4000`.

Endpoint y ejemplo de prueba (curl)

Envía un POST JSON a `/api/contact` con los campos requeridos.

```
curl -i -X POST http://localhost:4000/api/contact \
-H "Content-Type: application/json" \
-d
'{"nombre": "Prueba", "correo": "test@example.com", "telefono": "+56912345678", "mensaje": "Mensaje de prueba", "hp": ""}'
```

Respuesta esperada: `{"ok": true}` y en la consola del servidor `Mail sent:` con `info.accepted` mostrando los destinatarios.

Notas sobre recepción y forwarding

- Si usás un forwarder (p.e. `eforward*.registrar-servers.com`) el correo puede ser reenviado a una cuenta externa. Revisá el panel del registrador para logs y configuración de forwarding.
- En Gmail, si el email llega a la pestaña "Todos" y no genera notificaciones, creá un filtro por asunto/remitente y aplicá una etiqueta + activar notificaciones por etiqueta en la app móvil.

CC / BCC

- Puedes añadir `CC_EMAIL` o `BCC_EMAIL` en el `.env`. El servidor lee esas variables y las pasa a `nodemailer`.
- Para más robustez, en `server.js` se pueden convertir estas variables en arrays (`split(',').map(s=>s.trim())`) si querés validar múltiples direcciones.

Producción y buenas prácticas

- No poner `.env` en el repositorio; usar secretos del proveedor (Heroku, Vercel, Docker secrets, etc.).
- Configurar SPF/DKIM/DMARC si vas a enviar desde un dominio propio (no obligatorio si se usa Gmail SMTP, pero recomendado para evitar filtros si cambias a un SMTP del dominio).
- Monitorizar errores y agregar alertas (Sentry o similar) para fallos en el envío.

Qué hacer si hay problemas

- Revisá logs del servidor (la ruta `Mail sent:` y las variables `info.accepted` / `info.rejected` / `info.response`).

- Probar cambiando temporalmente `TO_EMAIL` a una cuenta que controles para aislar envío vs recepción.
-

Generado automáticamente como ayuda-memoria. Si querés, puedo añadir un script de prueba (`test-send.sh`) o convertir las direcciones CC/BCC a lista en `server.js`.